

MODULE MIA4 • JOUR 4 / 5

Performances & Extensions

Jour 4 — Aggregation Pipeline, GridFS, diagnostic & QCM

Requêtes analytiques puissantes • Stockage de fichiers • Index géospatiaux • Diagnostic, backup et sécurité

Plan de la journée

7 heures effectives · atelier aggregation + QCM d'évaluation

- 1 **Rappel Jour 3** — 9h00-9h30
- 2 **Aggregation Pipeline** — concepts et stages · 9h30-10h30
- 3 **Cas concrets d'agrégation** · 10h45-12h00
-  *Pause déjeuner — 12h00-14h00*
- 4 **GridFS & index géospatiaux** · 14h00-14h45
- 5 **Diagnostic : logs, mongostat, mongotop** · 14h45-15h30
- 6 **Backup, restauration & sécurité** · 15h30-16h15
- 7 **Atelier agrégation + QCM** · 16h15-17h00

HORAIRES

9h00 → 17h00

-  7h effectives
-  Pause 10h30-10h45
-  Déjeuner 12h-14h
-  QCM 30 min noté

ÉVALUATION

- ✓ QCM 30 questions
- ✓ Portée : Jours 1-4
- ✓ 30 min chrono
- ✓ TP individuel noté
- ✓ Sans documents

Rappel Jour 3 & ouverture Jour 4

🕒 Ce que nous avons couvert

- ✓ **Scale UP vs OUT** — MongoDB pense horizontal.
- ✓ **Replica Set** : primary + secondaries, oplog, heartbeats.
- ✓ **Élection** Raft — quorum majoritaire, nombre impair.
- ✓ **Write/Read Concern** : ajuster cohérence vs latence.
- ✓ **Sharding** : shard key = décision structurante.
- ✓ **Architecture cluster** : mongos, config server, shards.
- ✓ **Suivi** : distribution des chunks, targeted vs broadcast.

📈 Ouverture — Jour 4

Nous savons stocker, indexer, distribuer. Il reste à **exploiter** les données de manière analytique.

Le programme du jour

Aggregation Pipeline — le SQL GROUP BY à la puissance 10.

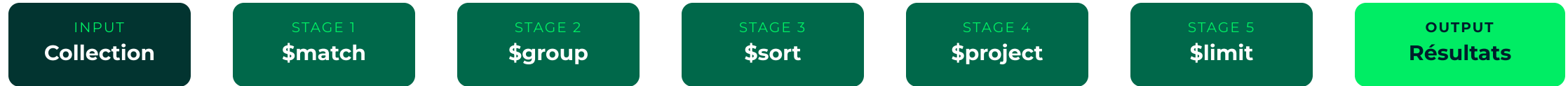
GridFS — stocker des fichiers > 16 Mo.

Diagnostic — comprendre ce qui se passe en production.

- » Pipelines de transformation type ETL
- » Requêtes géospatiales de proximité
- » Stockage de gros fichiers binaires
- » Backup, sécurité et bonnes pratiques

Aggregation Pipeline — le concept

Un flux de **stages** où chaque étape transforme le flux de documents et le passe au suivant — comme un pipeline Unix.



```
// Syntaxe générale
db.commandes.aggregate([
  // 1. Filtrer (comme WHERE en SQL)
  { $match: { annee: 2025 } },

  // 2. Grouper (comme GROUP BY)
  { $group: {
    _id: "$client_id",
    ca: { $sum: "$montant" },
    n: { $sum: 1 }
  }},

  // 3. Trier
  { $sort: { ca: -1 } },

  // 4. Top 10
  { $limit: 10 }
])
```

💡 Pourquoi c'est puissant

- Chaîne de **n stages** — pas de limite pratique
- Utilise les **index** quand le pipeline commence par **\$match**
- Support natif des **tableaux** avec **\$unwind**
- **Jointures** possibles via **\$lookup**
- Exportable via **\$out** ou **\$merge** vers une collection
- Traitement en **streaming** — peut manipuler des To

Les stages essentiels à connaître

Stage	Rôle	Équivalent SQL
\$match	Filtrer les documents — utilisez toujours en premier pour bénéficier des index.	WHERE
\$group	Regrouper par une clé, calculer \$sum , \$avg , \$min , \$max , \$push ...	GROUP BY
\$project	Choisir/renommer les champs, créer des champs calculés.	SELECT col1 AS x, ...
\$sort	Trier les documents. Utilise un index si placé au bon endroit.	ORDER BY
\$limit / \$skip	Pagination (attention à \$skip qui reste coûteux).	LIMIT / OFFSET
\$unwind	Éclater un tableau : 1 document par élément du tableau.	CROSS JOIN LATERAL
\$lookup	Joindre une autre collection — équivalent LEFT JOIN.	LEFT OUTER JOIN
\$addFields / \$set	Ajouter/mettre à jour des champs sans supprimer les autres.	SELECT *, col AS x
\$facet	Plusieurs sous-pipelines parallèles sur la même source — dashboard.	– (pas d'équivalent direct)
\$out / \$merge	Écrire le résultat dans une nouvelle collection (matérialisation).	CREATE TABLE AS / MERGE

Trois exemples pratiques d'agrégation

1 Top 5 clients — CA cumulé

```
db.commandes.aggregate([
  { $match: {
    annee: 2025,
    status: "payee"
  }},
  { $group: {
    _id: "$client_id",
    ca: {
      $sum: "$montant_ttc"
    },
    n_cmds: { $sum: 1 },
    panier_moy: {
      $avg: "$montant_ttc"
    }
  }},
  { $sort: { ca: -1 } },
  { $limit: 5 },
  { $lookup: {
    from: "clients",
    localField: "_id",
    foreignField: "_id",
    as: "client"
  }},
  { $project: {
    nom: {
      $first: "$client.nom"
    },
    ca: 1,
    panier_moy: 1
  } }
])
```

2 CA mensuel de l'année

```
db.commandes.aggregate([
  { $match: {
    date: {
      $gte: ISODate("2025-01-01"),
      $lt:  ISODate("2026-01-01")
    }
  }},
  { $group: {
    _id: {
      $dateTrunc: {
        date: "$date",
        unit: "month"
      }
    },
    ca: {
      $sum: "$montant_ttc"
    },
    nb: { $sum: 1 }
  }},
  { $sort: { _id: 1 } },
  { $project: {
    mois: "$_id",
    ca: 1,
    nb: 1,
    panier_moy: {
      $divide: ["$ca", "$nb"]
    },
    _id: 0
  } }
])
```

3 Top tags via \$unwind

```
db.articles.aggregate([
  // éclater le tableau tags
  { $unwind: "$tags" },

  // compter chaque tag
  { $group: {
    _id: "$tags",
    n: { $sum: 1 }
  }},

  { $sort: { n: -1 } },
  { $limit: 20 },

  // matérialiser le résultat
  { $merge: {
    into: "top_tags",
    whenMatched: "replace"
  } }
])

// Ensuite, top_tags contient
// la vue matérialisée.
// Idéal pour un dashboard.
```

Index géospatial — requêtes de proximité

L'index `2dsphere` permet des recherches géographiques précises (proche de X, dans un polygone...) sur des coordonnées GeoJSON.

📍 Modéliser une position

```
// Un boulanger à Paris — format GeoJSON
{
  nom: "Boulangerie Poilâne",
  position: {
    type: "Point",
    coordinates: [2.3298, 48.8503]
    // ↑ longitude  ↑ latitude
  }
}

// Créer l'index
db.boulangers.createIndex({position:"2dsphere"})
```

📍 Trouver les 5 plus proches

```
db.boulangers.find({
  position: {
    $near: {
      $geometry: {
        type: "Point",
        coordinates: [2.35, 48.85]
      },
      $maxDistance: 1000 // en mètres
    }
  }
}).limit(5)
```

📦 Opérateurs géospatiaux essentiels

`$near` · `$nearSphere` — les points proches d'une position, triés par distance croissante

`$geoWithin` — les points à l'intérieur d'une zone (polygone, cercle, boîte) — sans tri

`$geoIntersects` — les géométries qui intersectent avec une autre géométrie

`$centerSphere` — cercle défini par un centre + rayon en radians

Cas d'usage : Uber, Deliveroo, boutique-locator, "près de chez moi", géofencing IoT

GridFS — stocker de gros fichiers dans MongoDB

Rappel : un document BSON est limité à **16 Mo**. GridFS découpe les fichiers en **chunks** pour dépasser cette limite.

Fonctionnement — 2 collections

fs.files — un document de métadonnées par fichier : nom, taille totale, contentType, uploadDate, hash.

fs.chunks — les octets du fichier découpés en morceaux de 255 Ko par défaut. Chaque chunk a un `files_id` qui pointe vers `fs.files`.

Résultat : un fichier de 200 Mo génère ~800 chunks. MongoDB les recompose côté client à la lecture.

```
# PyMongo — upload et download d'un fichier
from pymongo import MongoClient
import gridfs

client = MongoClient("mongodb://localhost:27017")
fs = gridfs.GridFS(client["medias"])

# Upload
with open("cours-nosql.pdf", "rb") as f:
    file_id = fs.put(f, filename="cours-nosql.pdf",
                    contentType="application/pdf")

# Download
grid_out = fs.get(file_id)
with open("/tmp/copy.pdf", "wb") as f:
    f.write(grid_out.read())
```

Quand utiliser GridFS ?

Oui — fichiers > 16 Mo, versionnage, cohérence transactionnelle avec les données

Oui — infrastructure simple (une seule base à sauvegarder)

Non — CDN et servage HTTP à haut débit → S3 / MinIO

Non — fichiers < 16 Mo peuvent tenir dans un champ **Binary**

Logs MongoDB — la première source d'info

Depuis MongoDB 4.4, les logs sont au format **JSON structuré** — facilement ingérables dans ELK, Loki, Datadog.

⚙️ Configuration

```
# mongod.conf
systemLog:
  destination: file
  path: /var/log/mongodb/mongod.log
  logAppend: true
  verbosity: 0          # 0 (défaut) → 5
  component:
    query:      { verbosity: 1 }
    replication: { verbosity: 2 }
```

> Analyser en direct

```
# Live tail des queries lentes
tail -f /var/log/mongodb/mongod.log \
  | jq 'select(.attr.durationMillis>100)'

# Erreurs récentes
grep '"s":"E"' /var/log/mongodb/mongod.log

# Depuis le shell
db.adminCommand({ getLog: "global" })
```

🔔 Événements à monitorer en priorité

"c":"REPL" — élection en cours ou changement d'état de membre
"attr.durationMillis" — requêtes lentes (log auto au-delà de 100 ms)
"c":"STORAGE" — erreurs disque, saturation journal
"c":"NETWORK" — timeouts, disconnections répétées
"c":"ACCESS" — tentatives d'authentification échouées
"c":"SHARDING" — migrations de chunks, erreurs de balancer

mongostat & mongotop — top-like pour MongoDB

Deux utilitaires en ligne de commande installés avec les **MongoDB Database Tools**. Rafraîchissement en temps réel.

📊 mongostat — activité globale

```
# Toutes les 1s, host mongo1
$ mongostat --host mongo1:27017

insert query update delete getmore command dirty used flushes vsize  res  qrw arw net_in net_out conn
   15   210     8     2     0   421 0.5% 78%     0 2.1G 1.4G 0|0 0|0 42k  180k  67

# Colonnes clés :
# insert/query/update/delete = ops/sec
# dirty = % cache modifié en attente
# used  = % cache utilisé
# qrw   = requêtes en queue (r|w)
# conn  = connexions actives

# Alertes typiques :
# dirty > 20% → cache saturé, écritures lentes
# qrw > 0     → contention
# conn proche du max → augmenter maxIncomingConnections
```

📊 mongotop — quelles collections consomment

```
# Toutes les 2s
$ mongotop 2

ns                total    read    write
ecommerce.commandes 184ms   98ms   86ms
ecommerce.clients   92ms   82ms   10ms
ecommerce.produits  47ms   41ms    6ms
admin.system.replset  3ms    3ms    0ms

# Lecture :
# temps en ms passé par collection
# sur l'intervalle de rafraîchissement

# Repérer :
# - une collection qui monopolise
#   le temps CPU
# - un déséquilibre écriture/lecture
# - une collection inattendue
#   (indice de fuite applicative)

# Bonus : combiner avec Prometheus
# via mongodb_exporter pour
# l'historisation et les alertes.
```

Backup & Restauration MongoDB

Trois stratégies coexistent en production. La bonne stratégie combine les trois selon la RPO/RTO du métier.

1. mongodump / mongorestore

Export BSON logique. Idéal petites/moyennes bases (< 100 Go).

```
mongodump --uri=... \
--out=/backup/$(date +%F) \
--oplog # cohérent RS
```

2. Snapshot filesystem

Snapshot LVM, EBS, Azure Disk. Rapide, faible impact CPU.

```
db.fsyncLock()
aws ec2 create-snapshot ...
db.fsyncUnlock()
```

3. MongoDB Cloud / Ops Manager

Solution officielle avec restauration **Point-in-Time**. Grosses volumes / SLA critique.

Payant · MongoDB Atlas ou Ops Manager on-prem

```
# Restaurer la totalité
mongorestore --uri="mongodb://..." \
--drop # drop collections d'abord \
--oplogReplay # rejouer l'oplog \
/backup/2025-10-15/

# Une seule collection
mongorestore --nsInclude="ecommerce.clients" \
/backup/2025-10-15/

# Vers une autre base (renommer)
mongorestore --nsFrom="prod.*" \
--nsTo="staging.*" \
/backup/2025-10-15/
```

✔ Bonnes pratiques

- Backuper depuis un **secondary**, pas le primary
- Toujours **--oplog** sur un Replica Set (cohérence)
- **Chiffrer** les archives (gpg, KMS)
- Stocker **hors-site** (règle 3-2-1)
- **Tester la restauration** tous les 3 mois
- Documenter le **RPO** (perte tolérée) et **RTO** (temps de restauration)

Sécurité — les 6 fondamentaux

Historiquement, MongoDB a été fuité par des milliers d'entreprises pour un seul oubli : **l'authentification**. À ne jamais reproduire.



1. Authentification

Toujours activer `--auth`. Créer un compte admin dès l'installation. Aucune exception.



2. RBAC — rôles fins

Rôles built-in : **read**, **readWrite**, **dbAdmin**, **clusterAdmin**, **root**. Créer un rôle par appli, principe du moindre privilège.



3. Bind IP restrictif

Jamais `bindIp: 0.0.0.0` en prod sans firewall. Écouter uniquement le réseau applicatif interne.



4. TLS/SSL en transit

Chiffrer les communications client-serveur et intra-cluster. Certificats X.509 pour authentification mutuelle.



5. Encryption at rest

Storage engine encrypté (WiredTiger) en Enterprise, ou LUKS/BitLocker au niveau OS pour Community.



6. Audit & monitoring

Enterprise : `auditLog` des accès. Community : `rely on log` + tools externes (`auditd`, `Wazuh`).

```
// Créer un user applicatif avec droits limités
use ecommerce
db.createUser({
  user: "app_ecommerce",
  pwd: passwordPrompt(), // jamais en clair
  roles: [ { role: "readWrite", db: "ecommerce" } ],
  mechanisms: [ "SCRAM-SHA-256" ] // SHA-256 > SHA-1
})
```

TP INDIVIDUEL

TP noté — Aggregation, Geo & Diagnostic

☰ Consignes — sur le dataset ecommerce du Jour 2

- 1 **Aggregation** — CA mensuel de 2025 avec panier moyen, groupé par mois.
- 2 **\$lookup + \$unwind** — top 10 produits vendus avec le nom du produit.
- 3 **\$facet** — un pipeline qui retourne **en une seule requête** : CA total, nombre de clients uniques, top 5 catégories.
- 4 **Géospatial** — ajouter un champ `position` GeoJSON aux clients, créer l'index `2dsphere` . Requête : clients dans un rayon de 5 km autour de Paris.
- 5 **GridFS** — uploader 3 PDF, en télécharger 1, et lister les fichiers.
- 6 **Diagnostic** — activer le profiler, exécuter une requête lente volontaire, la retrouver dans `system.profile` .
- 7 **Backup** — mongodump de la base ecommerce, restauration dans `ecommerce_test` .
- 8 **Bonus** : créer un user `reader` ayant uniquement `read` sur ecommerce.

🚩 LIVRABLES

- `tp4.js` · scripts mongosh
- `geo.py` · script PyMongo geo
- `gridfs.py` · upload/download
- Capture d'`explain()`
- Rapport `bench.md`

🎓 ÉVALUATION (noté sur 20)

- Aggregations correctes /8
- Index geo + requête /3
- GridFS fonctionnel /3
- Diagnostic + profiler /2
- Backup/restauration /2
- Bonus user restreint /2

30 MIN

QCM d'évaluation — Jours 1 à 4

30 questions à choix multiples · sans documents · 30 minutes chrono · connexion Teams individuelle.

☰ Répartition des questions

J1 — Fondamentaux NoSQL5 questions

Big Data 5V · CAP · familles · JSON/BSN

J1 — MongoDB & CRUD5 questions

mongosh · insertOne · find · \$set · deleteMany

J2 — Modélisation & Index8 questions

Embed vs Ref · ESR · types d'index · explain · profiling

J3 — Repli & Sharding7 questions

Replica Set · élection · write concern · shard key

J4 — Agg & Perf5 questions

Pipeline · stages · GridFS · geo · sécurité

ℹ Modalités

- **Formulaire** Microsoft Forms
- **30 questions** · une seule réponse par question
- **Barème** : +1 bonne réponse · 0 blanc · 0 mauvaise
- Résultats disponibles en fin de journée
- Note finale = **QCM (40%) + TP jour (40%) + Projet (20%)**

💡 Conseils

- Lire chaque question **en entier** avant de répondre
- Attention aux formulations "sauf", "faux", "jamais"
- Ne pas s'attarder — 1 min max par question
- Revenir sur les incertitudes à la fin
- Vérifier les envois avant expiration

Ce qu'il faut retenir du Jour 4



Pipeline >> find

Toute requête analytique complexe passe par l'aggregation pipeline, jamais par plusieurs find.



\$match en premier

Placer \$match tôt permet d'utiliser les index et de réduire le volume envoyé aux stages suivants.



GeoJSON + 2dsphere

Coordonnées [lon, lat] · index 2dsphere · \$near / \$geoWithin. C'est natif et rapide.



GridFS > 16 Mo

Découpage automatique en chunks. Alternative simple à S3 pour beaucoup d'usages.



Diagnostic = 3 outils

Logs JSON (jq) · mongostat (activité) · mongotop (par collection). Compass pour visualiser.



Sécurité : jamais bindIp:0.0.0.0

Auth activée par défaut, RBAC granulaire, TLS, backup testé régulièrement.

→ **Demain — Jour 5** : le projet final. Vous concevrez et présenterez un **back-end WebApp** sur MongoDB. Prévoir vos idées de problématique.